



Assets

Managing assets in Odoo is not as straightforward as it is in some other apps. One of the reasons is that we have a variety of situations where some, but not all of the assets are required. For example, the needs of the web client, the point of sale app, the website or even the mobile application are different. Also, some assets may be large, but are seldom needed: in that case we may want them to be **loaded lazily (on demand)**.

Asset types [↗](#)

There are three different asset types: code (`js` files), style (`css` or `scss` files) and templates (`xml` files).

Code

Odoo supports **three different kinds of javascript files**. All these files are then processed (native JS modules are transformed into odoo modules), then minified (if not in `debug=assets` **mode**) and concatenated. The result is then saved as a file attachment. These file attachments are usually loaded via a `<script>` tag in the `<head>` part of the page (as a static file).

Style

Styling can be done with either `css` or **`scss`**[↗](#). Like the javascript files, these files are processed (`scss` files are converted into `css`), then minified (again, if not in `debug=assets` **mode**) and concatenated. The result is then saved as a file attachment. They are then usually loaded via a `<link>` tag in the `<head>` part of the page (as a static file).

Template

Templates (static `xml` files) are handled in a different way: they are simply read from the file system whenever they are needed, and concatenated.

Whenever the browser loads odoo, it calls the `/web/webclient/qweb/` controller to fetch the **templates**.

It is useful to know that in most cases, a browser only performs a request the first time it loads a page. This is because each of these assets are associated with a checksum, which is injected into the page source. The checksum is then added to the url, which means that it is possible to safely set the cache headers to a long period.

`js`, `css` or `scss`) is listed in the [module manifest](#). Files can be declared using [glob](#) syntax, meaning that you can declare several asset files using a single line.

The bundles are defined in each module's `__manifest__.py`, with a dedicated `assets` key which contains a dictionary. The dictionary maps bundle names (keys) to the list of files they contain (values). It looks like this:

```
'assets': {
    'web.assets_backend': [
        'web/static/src/xml/**/*.xml',
    ],
    'web.assets_common': [
        'web/static/lib/bootstrap/**/*.css',
        'web/static/src/js/boot.js',
        'web/static/src/js/webclient.js',
        'web/static/src/xml/webclient.xml',
    ],
    'web.assets_unit_tests': [
        'web/static/src/js/webclient.test.js',
    ],
},
```

Here is a list of some important bundles that most odoo developers will need to know:

- `web.assets_common` : this bundle contains most assets which are common to the web client, the website and also the point of sale. This is supposed to contain lower level building blocks for the odoo framework. Note that it contains the `boot.js` file, which defines the odoo module system.
- `web.assets_backend` : this bundle contains the code specific to the web client (notably the web client/action manager/views/static XML templates)
- `web.assets_frontend` : this bundle is about all that is specific to the public website: ecommerce, portal, forum, blog, ...
- `web.assets_unit_tests` : all javascript unit testing code (tests, helpers, mocks)

Operations

Typically, handling assets is simple: you just need to add some new files to a frequently used bundle like `assets_common` or `assets_backend`. But there are other operations available to cover some more specific use cases.

append

This operation adds one or multiple file(s). Since it is the most common operation, it can be done by simply using the file name:

```
'web.assets_common': [  
    'my_addon/static/src/js/**/*.js',  
],
```

By default, adding a simple string to a bundle will append the files matching the glob pattern at the end of the bundle. Obviously, the pattern may also be directly a single file path.

prepend

Add one or multiple file(s) at the beginning of the bundle.

Useful when you need to put a certain file before the others in a bundle (for example with css files). The `prepend` operation is invoked with the following syntax: `('prepend', <path>)`.

```
'web.assets_common': [  
    ('prepend', 'my_addon/static/src/css/bootstrap_overridden.scss'),  
],
```

before

Add one or multiple file(s) before a specific file.

Prepending a file at the beginning of a bundle might not be precise enough. The `before` directive can be used to add the given file(s) right *before* the target file. It is declared by replacing the normal path with a 3-element tuple `('before', <target>, <path>)`.

```
'web.assets_common': [  
    ('before', 'web/static/src/css/bootstrap_overridden.scss', 'my_addon/static/src.  
],
```

after

```
'web.assets_common': [  
    ('after', 'web/static/src/css/list_view.scss', 'my_addon/static/src/css/list_vi  
],
```

include

Use nested bundles.

The `include` directive is a way to use a bundle in other bundles to minimize the size of your manifest. In Odoo we use sub bundles (prefixed with an underscore by convention) to batch files used in multiple other bundles. You can then specify the sub bundle as a pair (`'include', <bundle>`) like this:

```
'web.assets_common': [  
    ('include', 'web._primary_variables'),  
],
```

remove

Remove one or multiple file(s).

In some cases, you may want to remove one or multiple files from a bundle. This can be done using the `remove` directive by specifying a pair (`'remove', <target>`):

```
'web.assets_common': [  
    ('remove', 'web/static/src/js/boot.js'),  
],
```

replace

Replace an asset file with one or multiple file(s).

Let us say that an asset needs not only to be removed, but you also want to insert your new version of that asset at the same exact position. This can be done with the `replace` directive, using a 3-element tuple (`'replace', <target>, <path>`):

Loading order

The order in which assets are loaded is sometimes critical and must be deterministic, mostly for stylesheets priorities and setup scripts. Assets in Odoo are processed as follows:

1. When an asset bundle is called (e.g. `t-call-assets="web.assets_common"`), an empty list of assets is generated
2. All records of type `ir.asset` matching the bundle are fetched and sorted by sequence number. Then all records with a sequence strictly less than 16 are processed and applied to the current list of assets.
3. All modules declaring assets for said bundle in their manifest apply their assets operations to this list. This is done following the order of modules dependencies (e.g. `web` assets is processed before `website`). If a directive tries to add a file already present in the list, nothing is done for that file. In other word, only the first occurrence of a file is kept in the list.
4. The remaining `ir.asset` records (those with a sequence greater than or equal to 16) are then processed and applied as well.

Assets declared in the manifest may need to be loaded in a particular order, for example `jquery.js` must be loaded before all other jquery scripts when loading the `lib` folder. One solution would be to create an `ir.asset` record with a lower sequence or a 'prepend' directive, but there is another simpler way to do so.

Since the unicity of each file path in the list of assets is guaranteed, you can mention any specific file before a glob that includes it. That file will thus appear in the list before all the others included in the glob.

```
'web.assets_common': [  
    'my_addon/static/lib/jquery/jquery.js',  
    'my_addon/static/lib/jquery/**/*',  
],
```

Note

A module *b* removing/replacing the assets declared in a module *a* will have to depend on it. Trying to operate on assets that have yet to be declared will result in an error.

a library once it is needed. To do that, the Odoo framework provides a few helper functions, located in `@web/core/assets`.

```
await loadAssets({
    jsLibs: ["/web/static/lib/stacktracejs/stacktrace.js"],
});
```

loadAssets(*assets*)

Arguments:

assets (`Object()`) – a description of various assets that should be loaded

Returns:

Promise<void>

Load the assets described by the `assets` parameter. It is an object that may contain the following keys:

Key	Type	Description
jsLibs	string[]	a list of urls of javascript files
cssLibs	string[]	a list of urls of css files

useAssets(*assets*)

Arguments:

assets (`Object()`) – a description of various assets that should be loaded

This hook is useful when components need to load some assets in their `onWillStart` method. It internally calls `loadAssets`.

The asset model (`ir.asset`)

In most cases the assets declared in the manifest will largely suffice. Yet for more flexibility, the framework also supports dynamic assets declared in the database.

This is done by creating `ir.asset` records. Those will be processed as if they were found in a module manifest, and they give the same expressive power as their manifest counterparts.

1. It provides a function returning a list of all file paths declared in a given list of addons (see `_get_addon_paths`);
2. It allows to create 'ir.asset' records to add additional directives to certain bundles.

name

Name of the asset record (for identification purpose).

bundle

Bundle in which the asset will be applied.

directive (default= `append`)

This field determines how the `path` (and `target` if needed) will be interpreted. Here is the list of available directives along with their required arguments:

- **append**: `path`
- **prepend**: `path`
- **before**: `target` , `path`
- **after**: `target` , `path`
- **include**: `path` (interpreted as a **bundle name**)
- **remove**: `path` (interpreted as a **target asset** to remove)
- **replace**: `target` , `path`

path

A string defining one of the following:

- a **relative path** to an asset file in the addons file system;
- a **glob pattern** to a set of asset files in the addons file system;
- a **URL** to an attachment or external asset file;
- a **bundle name**, when using the `include` directive.

target

Target file to specify a position in the bundle. Can only be used with the directives `replace` , `before` and `after` .

active (default= `True`)

Whether the record is active

sequence (default= `16`)

Loading order of the asset records (ascending). A sequence lower than 16 means that the asset will be processed *before* the ones declared in the manifest.

for conditional assets (for example, when a style option is enabled).

In such cases, the `<asset id="...">` syntax should be preferred over the `<record id="..." model="ir.asset">` syntax when defining the record in XML.

See **asset**, especially about the `active` value.

Get Help

[Contact Support](#)[Ask the Odoo Community](#)